

ECE391

Computer System Engineering

Lecture 25

Spring 2024

Slides Adapted from Prof. Kalbarczyk
University of Illinois at Urbana-Champaign

Lecture Topics

- **Linux Device Drivers**
 - Introduction and Abstractions
 - Basic Module Structure
 - Building and Loading Modules
 - Hello World Example
 - **Block** and **Character** Devices

Recap: Kernel Role in the System (1)

- ***Process management***
 - Implements the abstraction of several processes executing on top of a single/ multiple CPU(s).
 - E.g., the kernel is responsible for creating/destroying and scheduling processes.
- ***Memory management***
 - Supports virtual memory for all processes on top of the limited available resources.
- ***File systems***
 - Supports different file systems.
 - Almost everything in Linux can be treated as a file.

Device Drivers

- Q: what are differences between device drivers and regular user-space applications?
 - Runs in kernel space generally, access to hardware resources / ports.
 - Primarily deals with interaction between software and hardware, provides interface.
 - Runs on demand, depending on driver type and hardware needs. Loading a driver module isn't like executing a program.

Recap: Kernel Role in the System (2)

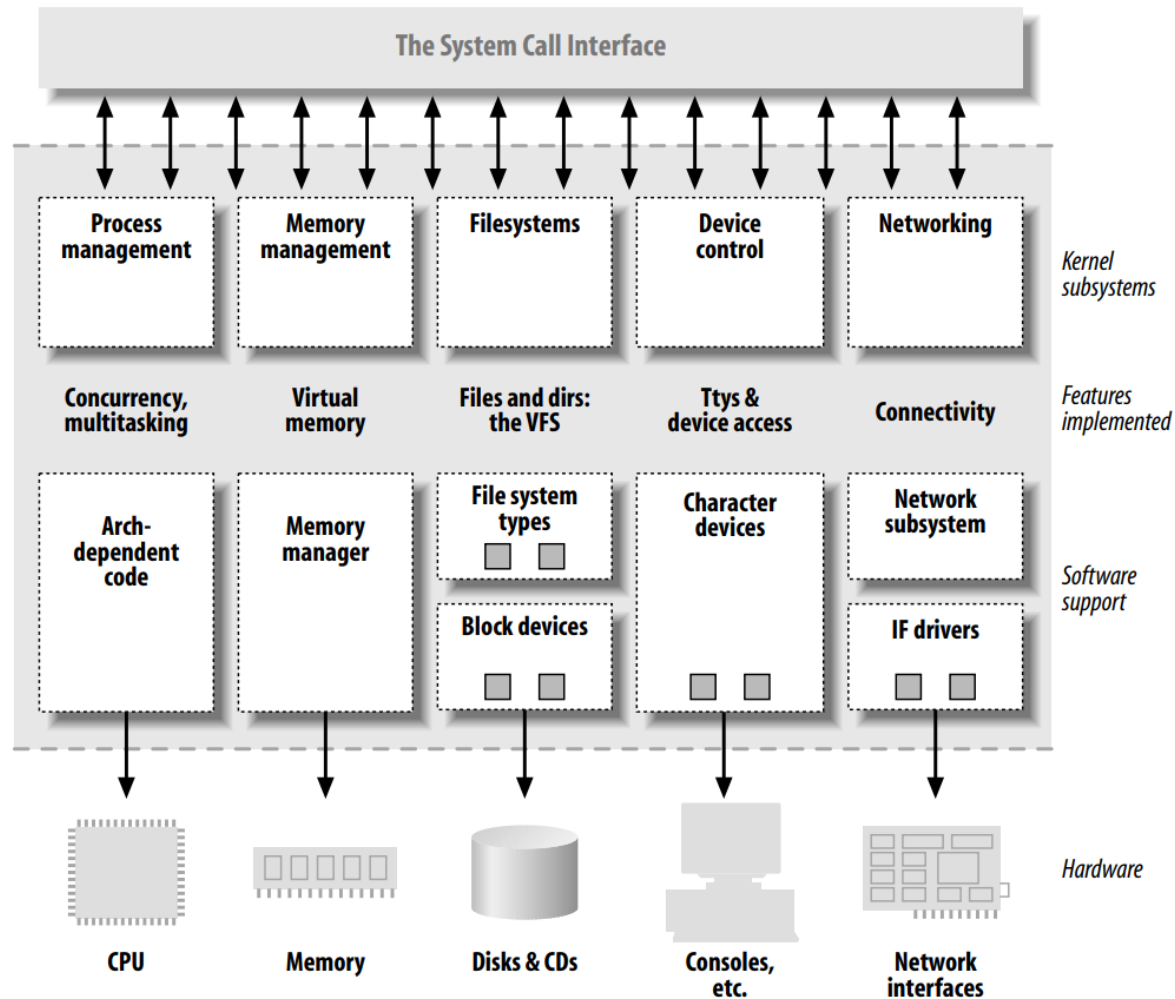
- ***Device control***

- Supports the system interactions with hardware devices.
- Almost every system operation eventually maps to a physical device.
- Device control operations are defined/implemented by a *device driver* specific to a given hardware device.

- ***Networking***

- Manages delivering data packets across program and network interfaces.
- Handles routing and address resolution.

Device Drivers in Linux OS



 *features implemented as modules*

Image: O'Reilly Media

Device Drivers



- Kernel interacts with I/O devices by means of device drivers.
 - Operate at the kernel level (**security is important**).
 - Include data structures and functions that control one or more devices, e.g., keyboard, monitors, network interfaces.
- Advantages:
 - Device code can be *encapsulated* in a module.
 - **Abstraction**: hide the details on how the device works.
 - Well-defined software-hardware interface.
 - Dynamic load/unload of device drivers.
 - Could also be permanently in kernel.

A Hello World Module

- Hello World module example: hello391.c:

```
#include <linux/init.h>
#include <linux/module.h>
```

Include kernel headers based on what your driver needs, should generally include at least `init.h` and `module.h`.

```
MODULE_LICENSE("Dual BSD/GPL");
MODULE_DESCRIPTION("ECE 391 Hello World Module Example");
MODULE_AUTHOR("UIUC");
```

Driver information:

- License
- Authors
- Description, etc.

```
static int hello_init(void) {
    printk(KERN_ALERT "Hello from ECE 391\n");
    return 0;
}
```

Init and exit functions, called on loading and unloading module (at the end of `init_module` system call), fixed format. See later example for passing parameters.

```
static void hello_exit(void) {
    printk(KERN_WARNING "Goodbye from ECE 391\n");
}
```

```
module_init(hello_init);
module_exit(hello_exit);
```

Defined in `<linux/module.h>`, lets kernel know which functions to call on loading and unloading module.

Building Modules

- Special Makefile format used for `kbuild`.
 - Make sure you have Linux kernel headers installed.
`$ sudo apt-get install linux-headers-$(uname -r)`
 - Read LDD3 pg. 23 “Compiling and Loading” for more details.
- Unlike regular binaries, kernel “objects” (.ko file) contains additional metadata about the module.
 - Kernel version to prevent compatibility issues.
 - Check with `modinfo` command.
`$ modinfo hello391.ko`
 - Additional file hash and kernel version information.
 - You should compile a driver from source (e.g., dkms).

Using insmod / rmmod

- Dynamically load a module with `insmod` command
 - Checks user permissions, file sanity, etc. (need **sudo**).
 - Then loads user space module (binary) into kernel space, resolving any symbols.
 - Internally makes an `init_module()` system call.
 - Finally, runs the module *init* function.

```
$ sudo insmod hello391.ko
```

```
$ dmesg
```

- Check currently loaded modules with `lsmod`.
- Remove module with `rmmod`.
 - Internally calls `delete_module()` system call.

```
$ sudo rmmod hello391
```

Hello World with Parameters

...

```
#include <linux/moduleparam.h>
```

...

```
static char *subject = "ECE";
```

```
static int classnum = 391;
```

```
module_param(classnum, int, S_IRUGO);
```

```
module_param(subject, charp, S_IRUGO);
```

Include headers for kernel symbols that you use.

```
#include <linux/moduleparam.h>
```

```
module_param(variable, type, perm);
```

Macro that creates a module parameter that can be adjusted by the user when the module is loaded (or at boot time for built-in code). The type can be one of bool, charp, int, invbool, long, short, ushort, uint, ulong, or intarray.

```
static int hello_init(void) {
```

```
    printk(KERN_ALERT "Hello from %s %d\n", subject, classnum);
```

```
    return 0;
```

```
}
```

S_IRUGO: set read permissions for owner user, group, other users (in this case, regarding modifying params).

We can now load with: `$ insmod param=value ...`

```
static void hello_exit(void) {
```

```
    printk(KERN_WARNING "Goodbye from %s %d\n", subject, classnum);
```

```
}
```

So far, we've only loaded / unloaded modules, no actual interaction with device files yet!

Kernel Abstraction for Devices – Device Files

- I/O devices are treated as special files called device files.
 - Same system calls used to interact with files on disk can be used to work with I/O devices.
 - E.g., same *write()* used to:
 - Write to a regular file **OR**
 - Send data to a printer by writing to */dev/lp0* device file.
- According to the characteristics of the underlying device drivers, device files can be of two types.
 - **Block**
 - **Character**

Kernel Abstractions for Devices:

Block Devices

- **Block devices** (underlies file systems):
 - Data accessible only in blocks of fixed size, with size determined by device.
 - Can be addressed randomly.
 - Transfers to/from device are usually buffered to and cached from for performance.
 - Examples: disks, CD ROM, DVD.

Kernel Abstractions for Devices:

Character Devices

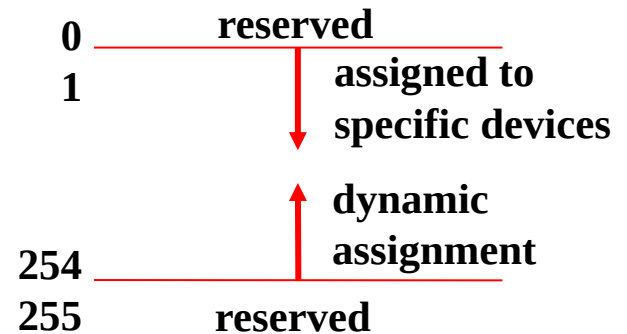
- **Character device:**
 - Almost everything else, except network cards.
 - Network cards are not directly associated with device files.
- **A more constructive definition:**
 - Contiguous space (or spaces) of bytes.
 - Some allow random access (e.g., magnetic tape driver).
 - Others available only sequentially (e.g., sound card).
 - Examples: keyboard, terminal, printers.

Kernel Abstractions for Devices

- A device file is usually stored as a real file:
 - **Inode** includes an identifier of the hardware device corresponding to the character or block device file.
- Devices identified by **major** and **minor** numbers (traditionally both 8-bit (256), now larger 12-bit + 20-bit in Kernel 2.6).
 - Major number is device type (corresponds to driver).
 - Minor number is instance number (if driver allows you to have more than one of a given model attached to your computer).
 - Associates devices with their corresponding drivers.
 - You can check with `ls -l` in `/dev`.
 - Also, check registered devices in `/proc/devices`.

Kernel Abstractions for Devices

- Major numbers each have:
 - associated `fops` structure.
 - name (see `/proc/devices`).



Example of device files:

Name	Type	Major	Minor	Description
/dev/hda	block	3	0	First IDE disk
/dev/hda2	block	3	2	Secondary primary partion of first IDE disk
/dev/tty0	char	3	0	Terminal

Device Driver Registration

- Each system call issued on a device file is translated by the kernel into an invocation of a function of a corresponding device driver.
- To achieve this, a device driver must register itself:
 - A device driver statically compiled in the kernel is registered during the kernel initialization phase .
 - A device driver compiled as a kernel module is registered when the module is loaded.
 - Note that the driver must register its own devices.
 - ***insmod*** e.g., `insmod ./hello391.ko` #link the module to the running kernel
 - ***rmmod*** e.g., `rmmod ./hello391` #unlink the module from the running kernel

Kernel Abstractions for Devices

- Registering and unregistering a device (see linux/fs.h).

```
int register_chrdev (unsigned int major,  
                    const char* name, struct file_operations* fops);
```

- To request a specific major #, pass it as input argument
 - Returns 0 on success.
 - Returns negative value on failure.
- For a dynamically assigned major #, pass 0 as input argument (major)
 - Returns assigned major # on success (not 0!).
 - Returns negative value on failure.

Kernel Abstractions for Devices

```
int unregister_chrdev (unsigned int major,  
                      const char* name);
```

- Both parameters must match those of registration call.
- Returns 0 on success.
- Returns negative value on failure.

Hello World with Device Registration

```
...
int hello_major = 0;
static int hello_init(void) {
    printk(KERN_ALERT "Hello from ECE 391");
    int result;
    // Register your major, and accept a dynamic number
    result = register_chrdev(hello_major, "hello", &hello_fops);
    if (result < 0)
        return result;
    if (hello_major == 0)
        hello_major = result; /* dynamic */
    return 0;
}
static void hello_exit(void) {
    unregister_chrdev(hello_major, "hello");
    printk(KERN_WARNING "Goodbye from %s %d\n", subject, classnum);
}
```

Important Data Structures Used by the Driver

- Driver operations involve use of three kernel data structures:
 - ***File operations***
 - Jump table of functions in the driver that implement specific file operations.
 - ***File structure (or file object)***
 - Represents an open file.
 - No image in the permanent storage.
 - Created by the kernel on “open” system call.
 - ***Inode***
 - Used internally by the kernel to represent file metadata.
 - Store in the permanent storage.

File Operations

- File Operations Structure:
 - Jump table of file operations / character driver operations.
 - Generic instance for files on disk.
 - Distinct instances for sockets, etc.
 - One instance per device type.
- Provide open, read, write, close, etc. functions.